

# Python Module 2 - Collections, Functions and Modules

## Complete Detailed Interview Handbook (40 Questions)

### ***1. What is a Python list, and how can it store values of different data types?***

A list is an ordered, mutable collection. It can store multiple data types together.

Example: data=[10,'Python',5.5,True]

### ***2. How would you update the value at a specific index in a list?***

```
nums=[10,20,30]
nums[1]=50
```

### ***3. Describe two ways to remove an element from a list in Python.***

remove(value) removes by value.

del list[index] or pop(index) removes by index.

### ***4. How does a for loop help you iterate through all elements in a list?***

```
for item in mylist:
    print(item)
```

### ***5. What is the purpose of using range with lists in a for loop?***

range() provides indexes.

```
for i in range(len(mylist)):
    print(i,mylist[i])
```

### ***6. Write a Python loop to print each element in the list ['a',3,5.5,'hello'] along with its index.***

```
lst=['a',3,5.5,'hello']
for i,v in enumerate(lst):
    print(i,v)
```

### ***7. What does round() do when applied to float values?***

round() rounds decimal values.

```
nums=[2.3,4.8]
print([round(x) for x in nums])
```

### ***8. Difference between sort() and sorted()?***

sort() changes the original list. sorted() returns a new sorted list.

### ***9. Use zip() to combine two lists into tuples.***

```
a=['A','B']  
b=[1,2]  
result=list(zip(a,b))
```

**10. What is a tuple, and how is it different from a list?**

Tuple is immutable; list is mutable.

**11. Create a tuple containing strings and numbers.**

```
t=('Python',101,5.5)
```

**12. Explain tuple slicing.**

```
t=(10,20,30,40)  
print(t[1:3])
```

**13. Convert tuple to list and vice versa.**

```
t=(1,2,3)  
l=list(t)  
t=tuple(l)
```

**14. Update a dictionary value.**

```
student={'age':20}  
student['age']=21
```

**15. What is a nested dictionary?**

```
student={'Ravi':{'Math':90,'Science':85}}
```

**16. Create a dictionary using zip().**

```
keys=['name','age']  
values=['Ravi',21]  
d=dict(zip(keys,values))
```

**17. Create a dictionary using a loop.**

```
d={}  
for i in range(len(keys)):  
d[keys[i]]=values[i]
```

**18. Count character frequency using a dictionary.**

```
text='hello'  
freq={}  
for ch in text:  
freq[ch]=freq.get(ch,0)+1
```

**19. Program to count words in a sentence.**

```
s=input('Enter sentence: ')
print(len(s.split()))
```

### ***20. Difference between positional and keyword arguments.***

Positional arguments depend on order. Keyword arguments use parameter names.

### ***21. Set a default value for a function argument.***

```
def greet(name='Guest'):
    print(name)
```

### ***22. Function with default second argument 10.***

```
def add(a,b=10):
    return a+b
```

### ***23. What is a lambda function?***

A lambda is an anonymous one-line function.

### ***24. Lambda function returning product.***

```
mul=lambda a,b:a*b
```

### ***25. Sort tuples by second element.***

```
data=[('A',3),('B',1)]
print(sorted(data,key=lambda x:x[1]))
```

### ***26. Create a custom module.***

```
student.py
```

```
def show():
    print('Hello')
```

### ***27. Call function from module.***

```
import student
student.show()
```

### ***28. Name two math module functions.***

math.sqrt(x) and math.factorial(x)

### ***29. Generate a random integer between 1 and 100.***

```
import random
print(random.randint(1,100))
```

### ***30. Purpose of \_\_init\_\_.py.***

It marks a folder as a Python package.

### **31. Structure a Python package.**

```
mypackage/  
__init__.py  
module1.py  
module2.py
```

### **32. List comprehension for squares 1 to 5.**

```
squares=[x*x for x in range(1,6)]
```

### **33. Flatten a 2D list using nested comprehension.**

```
matrix=[[1,2],[3,4]]  
flat=[x for row in matrix for x in row]
```

### **34. Dynamic nested dictionary.**

```
students={'Ravi':{'Math':90,'Science':80}}
```

### **35. Add a new subject and mark.**

```
students['Ravi']['English']=88
```

### **36. Using AI tools like ChatGPT or Copilot.**

Use them for code generation and debugging, but always test and verify outputs.

### **37. Generate a word-frequency function with AI.**

Ask AI to generate code, then test with sample inputs and edge cases.

### **38. Function mapping sentences to word counts.**

```
def word_counts(sentences):  
    result={}  
    for s in sentences:  
        result[s]=len(s.split())  
    return result
```

### **39. Create student-mark dictionary and find topper.**

```
names=['A','B']  
marks=[80,95]  
d=dict(zip(names,marks))  
print(max(d,key=d.get))
```

### **40. Sort list of dictionaries by age.**

```
students=[{'name':'A','age':22},{'name':'B','age':20}]  
print(sorted(students,key=lambda x:x['age']))
```